

Software Development Life Cycle (SDLC) Policy

Langate Group

Purpose

This policy defines the high-level requirements for providing business program managers, business project managers, technical project managers, and other program and project stakeholders guidance to support the approval, planning, and life-cycle development of Langate Group software systems aligned with the Information Security Program.

Roles and Responsibilities

- The **Delivery Director** is responsible for maintaining and implementing the "Software Development Process and Standard Approach" part of this policy.
- **Chief Technology Officer** (CTO) is responsible for maintaining and implementing the "Secure System Engineering Principles and SDLC Security Control Guidelines" part of this policy.

Policy

Langate Group must establish and maintain processes to ensure its computer applications or systems follow a consistent and repeatable SDLC process and maintain information security at every stage.

Software Development Process and Approach Standard

A PMO (Project Management Office) is a group within an organization responsible for coordinating and managing Software Development Projects. The PMO process is a set of guidelines and procedures that the Project Manager (PM) uses to ensure that all projects within Langate are executed in a consistent and efficient manner. The purpose of the PMO process is to provide a structured approach to project management and to ensure that all projects are aligned with Langate Group's goals and objectives. The PMO process also helps to ensure that all projects are completed on time, within budget, and to the satisfaction of all stakeholders.

Project Management Office (PMO) Process Steps:

Project Initiation

Project initiation is the first step in the PMO process, and it is critical to ensure that the project is properly defined and aligned with the organization's goals and objectives. The project initiation process is used to determine the feasibility of a project, identify the project's stakeholders, and develop a project charter and a project plan. The purpose of project initiation is to establish a clear understanding of the project's objectives, scope, and deliverables. The project initiation process also helps to identify any potential risks and to establish a clear project plan.

Project Planning

Project planning is the second step in the PMO process, and it is critical to ensure that the project is properly planned and executed. The project planning process is used to develop a detailed project plan, including tasks, timelines, and resource requirements. The purpose of project planning is to develop a clear understanding of the project's objectives, scope, and deliverables and to ensure that the project is executed in a consistent and efficient manner. Project planning also helps to identify potential risks and to develop a risk management plan.

Project Execution and Monitoring

Project execution and monitoring is the third step in the PMO process, and it is critical to ensure that the project is progressing as planned and that any issues or deviations are identified and addressed in a timely manner. The project monitoring and control process is used to track the project's progress and performance and identify any areas that require adjustments to keep the project on track. Project monitoring and control also help to ensure that the project is executed within budget and that the project's objectives are met.

Project Closure

Project closure is the final step in the PMO process, and it is critical to ensure that the project is properly closed out and that all deliverables are handed over to the appropriate stakeholders. The project closure process is used to close the project formally and conduct a post-project review to identify any lessons learned and make recommendations for future projects.

Project Management

The sequence of the development phases depends on the software development approach taken. The project management approaches include but are not limited to:

- Waterfall Development
- Agile Development
- Iterative Development
- Increment Development

In Iterative Development, there may be multiple Cycles (iterations) of the above phases before the final software is released.

Secure System Engineering Principles

Information security implications will be addressed and reviewed regularly, and responsibilities for information security will be defined and allocated to the roles defined in the project management methods.

Engineering principles for a secure system will fall under the following categories:

- Business Layer
 - Access to the business functions should be based on the authentication level.
 - There should be no "super-admin" backdoor functionality with full system access.
 - Permissions must be assigned on a group level and preferably never on a user level.
- Data Layer
 - Use impersonated authentication into a database when it's possible versus username+password.
 - Do not provide sysadmin database permissions to the application's account.
 - Do not share or do not allow individuals to log into the staging/production database using the application's credentials.
 - Use data encryption at rest if the database engine supports it.
 - By default, use data masking functionality for sensitive data if the database engine supports it.
 - If a dev team member needs access to the stage/production database, always create a personalized account versus using shared credentials.
 - Individual user accounts should get access to the database via group membership instead of individual security configuration.
 - Login security audit must be enabled in the production environments.
- Applications
 - Keep third-party libraries and dependencies up to date to patch known security vulnerabilities.
 - Follow secure coding standards and guidelines for the programming language and framework you are using.

- Input validation: always validate and sanitize input data to protect against common attacks like SQL injection, cross-site scripting (XSS), and command injection.
- Implement CSRF protection mechanisms, such as anti-CSRF tokens, to prevent attackers from executing actions on behalf of authenticated users.
- Use input validation libraries and built-in validation functions where available.
- Output Encoding: encode output data to prevent XSS attacks. HTML, XML, and URL encoding are common techniques.
- For session cookies, use HttpOnly, Secure.
- Set appropriate security headers in HTTP responses, including Content Security Policy (CSP), X-Content-Type-Options, X-Frame-Options, and X-XSS-Protection.
- Implement proper error handling to avoid exposing sensitive information in error messages.
- Log errors securely, and do not log sensitive data like passwords, user details, or PHI.

SDLC Security Control Guidelines

The SDLC process will adhere to the following information security controls:

- Adequate procedures should be established to provide separation of duties in the origination and approval of source documents. This shall include but not be limited to separation of duties between Personnel assigned to the development/test environment and those assigned to the production environment.
- Modification of code or an emergency release will follow the change control standard.
- Secure programming standards should be followed. Secure code training should be provided to Langate Group's developers.
- A Secure development environment will be created based on:
 - sensitivity of data to be processed, stored, and transmitted by the system;
 - applicable external and internal requirements, e.g., from regulations or policies;
 - security controls already implemented by the organization that support system development;
 - trustworthiness of personnel working in the environment;
 - the degree of outsourcing associated with system development;
 - the need for segregation between different development environments;
 - control of access to the development environment;
 - monitoring of change to the environment and code stored therein;
 - backups are stored at secure offsite locations; and,
 - control over movement of data from and to the environment.
- Threat modeling, incident reviews, use of vulnerability thresholds, or contingency planning will be conducted to ensure the architecture and design of information systems are protected against known threats based on the operational environment.
- All software deployed on Corporate or Hosted infrastructure must prevent security issues, including but not limited to those covered by OWASP.
- Code changes are reviewed by individuals other than the originating code author and by individuals who are knowledgeable in code review techniques and secure coding practices.
- Overrides of edit checks, approvals, and changes to confirmed transactions should be appropriately authorized, documented, and reviewed.
- Application development activity should be separated from the production and test environments. The extent of separation, logical or physical, is recommended to be appropriate to the risk of the business application or be in line with customer contractual requirements. The level of separation that is necessary between production, development, and test environments should be evaluated, and controls should be established to secure that separation.
- Test environments are designed to match production environments as closely as possible.
- All changes to production environments should strictly follow change control procedures, including human approval of all changes, granted by an authorized owner of that environment. Automated updates should be disallowed without such approval.

Individuals who are responsible for supporting or writing code for an internet-facing application, or internal application

that utilizes web technology and handles customer information, should complete **annual** security training specific to secure coding practices. For individuals supporting or writing code for an internet-facing application, training should also include topics specific to internet threats. The individual should complete the training prior to writing or supporting the code. The training must include OWASP secure development principles.

- Custom accounts and user IDs and/or passwords should be removed from applications before applications become active or are released to customers.
- Production data should not be used in testing (except in situations where it is explicitly required) or development environments.
- Security controls that are in place for the production copy in the test system should be production quality (e.g. mirroring the production controls over the data).
- When conducting quality assurance (QA) testing prior to the release of a new feature requiring user input where constraints on user input may be reasonably understood, feature acceptance tests must include testing of edge and boundary cases.

For situations demonstrating that testing needs to use production data, the requirements are the following:

- The Information Resource Owner will provide approval before production data can be used for testing purposes.
- Wherever possible, the production data should be tokenized, or anonymized, removed, or de-identified instead of using production data.
- Testing and parallel runs should use a separate copy of production data, and the test location or destination should be acceptable (e.g., loading confidential production data to a laptop for testing is not acceptable).
- The data should not be extracted, handled, or used by the test process in a manner that subjects the data to unauthorized disclosure.
- The data should be accessed on a need-to-know basis with the same access control procedures as operational environments.
- The data should have a separate authorization each time operational information is copied to a test environment.
- The fact of copying data into the test environment should be logged for an audit trail.
- Normal test activities should not use production data. In cases where test activity requires access to production data, access to production data should be restricted to only those individuals who have a documented business need. Only the documented business needs information should be accessible to those users.
- Production data used for testing should be securely erased upon completion of testing.
- Test data and accounts will be removed before being placed into production.
- Restricted/Protected Information will be encrypted according to the Encryption Standard while at rest or in transit.
- Error messages must be handled securely, and they must not leak sensitive information.

If production is outsourced, the following will be considered, as applicable, across the entire external supply chain in conjunction with the requirements outlined in the *Vendor Management Policy*:

- Licensing arrangements, code ownership, and intellectual property rights related to the outsourced content.
- Contractual requirements for secure design, coding, and testing practices.
- Provision of the approved threat model to the external developer.
- Acceptance testing for the quality and accuracy of the deliverables.
- Provision of evidence that security thresholds were used to establish minimum acceptable levels of security and privacy quality.
- Provision of evidence that sufficient testing has been applied to guard against the absence of both intentional and unintentional malicious content upon delivery.
- Provision of evidence that sufficient testing has been applied to guard against the presence of known vulnerabilities.
- Escrow arrangements, e.g., if source code is no longer available.
- Contractual right to audit development processes and controls.
- Effective documentation of the built environment used to create deliverables.

- The organization remains responsible for compliance with applicable laws and control efficiency verification.

Policy version history

 Policy details

Policy name	Renewal date
Software Development Life Cycle Policy	September 26, 2026
Description	
This policy defines the high-level requirements for providing business program managers, business project managers, technical project managers, and other program and project stakeholders guidance to support the approval, planning, and life cycle development of the company's software systems.	

 Version history

Version 1					
Creation date	Approval date	Published date	Owner	Approver(s)	Publisher
April 2, 2024	April 17, 2025	April 17, 2025	Eugene Skubach	Eugene Skubach	Eugene Skubach
Version 1.1					
Creation date	Approval date	Published date	Owner	Approver(s)	Publisher
May 6, 2025	No approval required	May 6, 2025	Eugene Skubach	-	Eugene Skubach
Version 1.2					
Explanation of changes					
'Revision History' section has been removed					
Creation date	Approval date	Published date	Owner	Approver(s)	Publisher
September 26, 2025	No approval required	September 26, 2025	Eugene Skubach	-	Eugene Skubach